Práctica 2: DQN y Rainbow para locomoción visual en control continuo discretizado

Antonio Lorenzo Andrés Martínez Pablo García

Curso 2025/2026

Resumen

En esta práctica estudiamos cómo se comportan DQN y Rainbow en una tarea de locomoción visual dentro de MuJoCo. Para ello entrenamos ambos métodos sobre el mismo entorno, `Walker2d-v4`, manteniendo fijos tanto la discretización del espacio de acciones como el preprocesado de las imágenes y la función de recompensa. Además, realizamos un estudio de ablación sobre dos componentes de Rainbow, *Distributional RL* y *Prioritized Experience Replay* (PER), para ver hasta qué punto afectan al aprendizaje, a la estabilidad del entrenamiento y al rendimiento final.
Como el entorno original tiene acciones continuas, fue necesario transformarlo en un problema discreto con un conjunto reducido de acciones. El input del agente se construyó a partir de imágenes RGB redimensionadas y apiladas temporalmente. Los resultados muestran que Rainbow puede superar claramente a la DQN base, aprendiendo más rápido y alcanzando recompensas bastante más altas. Sin embargo, también vimos que Rainbow es más sensible y requiere un ajuste más fino de hiperparámetros para funcionar bien.
En el estudio de ablación se observa que el componente distribucional es el que más peso tiene en este entorno, mientras que PER también ayuda, pero de una forma menos decisiva. En conjunto, los experimentos muestran que en este tipo de tareas no solo nos importa el algoritmo, sino que también el pipeline y las configuraciones de hiperparámetros son relevantes.

1. Introducción

DQN fue uno de los primeros algoritmos de Deep Reinforcement Learning capaces de aprender directamente a partir de imágenes, aproximando la función de valor de acción mediante una red neuronal convolucional entrenada con Q-learning [1]. A partir de esta idea, Rainbow propone ir un paso más allá combinando varias mejoras que habían ido apareciendo sobre DQN, como Double DQN, *dueling networks*, *multi-step returns*, *Noisy Nets*, *Prioritized Experience Replay* y aprendizaje distribucional [2]. La idea es sencilla: mantener la base de DQN, pero juntar en un solo modelo técnicas que por separado ya habían demostrado funcionar bien.

En esta práctica el problema es más exigente que los casos clásicos de Atari. Por un lado, el entorno de MuJoCo tiene un espacio de acciones continuo, mientras que DQN y Rainbow están pensados para acciones discretas. Por otro, el agente no recibe directamente el estado físico del sistema (como un vector de estado), sino únicamente imágenes RGB del entorno. Esto hace que tengamos que resolver dos problemas a la vez: aprender una representación visual útil y, sobre esa representación, aprender a controlar el movimiento.

Para poder aplicar estos métodos, fue necesario construir un pipeline propio y tomar varias decisiones de diseño importantes. En primer lugar, discretizamos manualmente el espacio de acciones para adaptar el problema a DQN y Rainbow. En segundo lugar, utilizamos únicamente observaciones visuales como entrada, sin añadir información propioceptiva. Por último, introdujimos una pequeña modificación en la recompensa mediante *reward shaping*, con el objetivo de facilitar el aprendizaje sin cambiar la meta general de avanzar hacia delante.

El objetivo del trabajo ha sido comparar de forma experimental DQN y Rainbow en una tarea de locomoción sobre `Walker2d-v4` y, después, analizar qué ocurre al eliminar dos componentes concretos de Rainbow: la parte distribucional y PER. Más allá de ver qué método obtiene más recompensa, nos interesaba entender mejor qué partes del modelo explican realmente la mejora y cuáles ayudan más bien a hacer el entrenamiento más estable o más eficiente.

2. Parte 1: Locomoción con DQN

2.1. Metodología

Se eligió el entorno `Walker2d-v4`, uno de los entornos sencillos propuestos en la práctica. Aun siendo más manejable que `Humanoid`, seguía siendo un problema exigente porque el agente debía aprender a andar a partir de imágenes y sobre un entorno originalmente continuo.

La discretización del espacio de acciones fue la decisión de diseño más importante. En lugar de discretizar cada dimensión mediante un producto cartesiano completo, se

definió un conjunto pequeño de acciones prototipo. Con `num_bins=3`, el *wrapper* genera una acción nula y, para cada articulación, dos acciones no nulas que actúan solo sobre esa dimensión. Como `Walker2d` tiene seis dimensiones de acción, el número final de acciones discretas es

$$1 + 6 \cdot (3 - 1) = 13.$$

Esta elección convierte el problema en algo tratable para DQN, aunque limita el control tan fino que podría tener sin estar discretizado y obliga al agente a aprender a moverse encadenando varios impulsos simples.

La observación se consigue a partir del `render` RGB del entorno. Cada *frame* se recorta, se redimensiona a 84×84 y se apilan cuatro imágenes consecutivas, de forma que la entrada final tiene 12 canales. Se decidió mantener color RGB en lugar de pasar a escala de grises porque, en este entorno, la información visual sobre postura y movimiento de las piernas, sobretodo el poder distinguir una pierna de la otra, nos pareció útil.

La recompensa usada en `Walker2d` fue

$$r_t = r_t^{\text{orig}} + 0{,}1\, qvel_x,$$

es decir, la recompensa original del entorno más un término proporcional a la velocidad horizontal. La idea detrás de esta decisión fue dar un bonus pequeño que motive el avance hacia delante sin imponer explícitamente una forma concreta de caminar ni tocar demasiado la función de recompensa original.

La arquitectura de DQN siguió el patrón clásico tipo Nature DQN, inspirado en la propuesta original de Mnih et al. [1]. El Encoder convolucional consta de tres capas `Conv2d` con 32, 64 y 64 filtros, núcleos $8, 4, 3$ y *strides* $4, 2, 1$, seguido de una MLP con una hidden layer de 512 unidades y una salida lineal con un valor Q por acción discreta. El entrenamiento se hizo con *experience replay*, red objetivo y exploración ε-greedy, con ε desde 1.0 hasta 0.01 y una constante de decaimiento de 100 000 pasos.

Cuadro 1: Configuración principal del experimento.

Parámetro	Valor
Entorno	`Walker2d-v4`
Observación	RGB
Resolución	84×84
Frame stack	4
Canales de entrada	12
Pasos totales	20 000 000
Tamaño de buffer	100 000
Batch size	32
Learning rate	10^{-4}
γ	0.99
Target update	5 000 pasos
Learning starts	1 000 pasos
Train frequency	4 pasos
Gradient clipping	10.0
$\varepsilon_{\text{start}}$	1.0
ε_{end}	0.01
ε-decay	100 000

Cuadro 2: Diseño del espacio de acciones y del preprocesado visual.

Elemento	Configuración
`num_bins`	3
`action_scale`	1.0
Dimensión de acción continua	6
Acciones discretas finales	13
Crop visual	top 0.25, bottom 0.05 left 0.10, right 0.10
Healthy angle range	$(-0{,}4,\ 0{,}4)$
Reward shaping	$r^{orig} + 0{,}1\, qvel_x$

De la Tabla 1 conviene destacar, sobre todo, que el entrenamiento duró 20 millones de pasos, ya que en este tipo de problemas en el que aprendes de imágenes el coste muestral es alto y con presupuestos más pequeños de pasos no aprendía demasiado. También es relevante haber usado un *frame stack* de 4 imágenes, porque con una sola observación el agente tendría mucha menos información sobre velocidad y dinámica, algo especialmente importante en una tarea de locomoción.

Por otro lado, la Tabla 2 resume probablemente las decisiones de diseño más delicadas de toda la práctica. Como ya se ha nombrado antes, la más importante fue la discretización del espacio de acciones: pasar de un control continuo de 6 dimensiones a solo 13 acciones discretas hizo que la DQN pueda abordar el problema, aunque a costa de restringir bastante el control tan fino del agente. También fue importante el *reward shaping* basado en la velocidad horizontal, ya que ayudó a reforzar el comportamiento deseado (que vaya hacia delante) sin cambiar el objetivo general de la tarea.

2.2. Resultados y discusión

Para DQN realizamos dos ejecuciones completas y ambas consiguieron aprender una política mejor que la inicial. En la Figura 1 se observa que no comienza a aprender de forma inmediata, sino que durante los primeros pasos la recompensa se mantiene baja. A partir de ahí aparece una fase de mejora clara en ambas runs, lo que indica que el agente empieza a aprovechar correctamente la discretización y la señal de recompensa.

Como métrica principal se ha utilizado `Reward/MeanWindow`, ya que reduce bastante la variabilidad entre episodios. En la primera ejecución, DQN alcanza un valor final de 773.57, mientras que en la segunda llega a 869.45. Aunque se ve cierta diferencia entre ambas, las dos curvas evolucionan de forma bastante parecida y se mantienen en el mismo orden de magnitud, lo que nos sugiere que el comportamiento del agente es relativamente consistente.

En cuanto a la dinámica del aprendizaje, en los dos casos se ve una mejora rápida al inicio, pero en las siguientes fases hay una zona más inestable donde la recompensa oscila sin crecer de una manera clara. Esto sugiere que DQN puede encontrar pronto una estrategia para moverse de forma razonable, pero le cuesta consolidarla y mejorarla a largo plazo. De hecho, aunque aparecen episodios con recompensas altas, estos no se mantienen de forma estable en el tiempo.

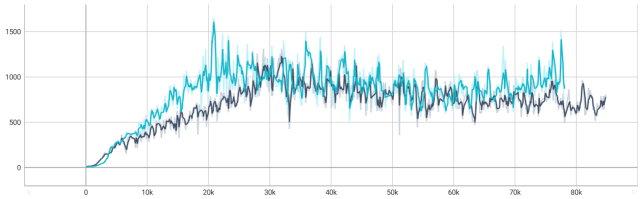


Figura 1: Curva `Reward/MeanWindow` de las dos ejecuciones de DQN.

Las pérdidas TD (Figura 2) muestran un comportamiento bastante ruidoso, con picos frecuentes a lo largo de todo el entrenamiento. Aunque el nivel medio está detro de unos limites razonables, la presencia de estos picos tan marcados indica que el agente sigue encontrando transiciones difíciles incluso en fases avanzadas. Aun así, no se observa una divergencia clara, lo que sugiere que el entrenamiento se mantiene estable en términos generales.

Además, este patrón se repite en ambos runs, lo que refuerza la idea de que el ruido en la estimación de valores es una característica propia del problema, probablemente debido al uso de observaciones visuales y a la simplificación del control continuo mediante discretización.

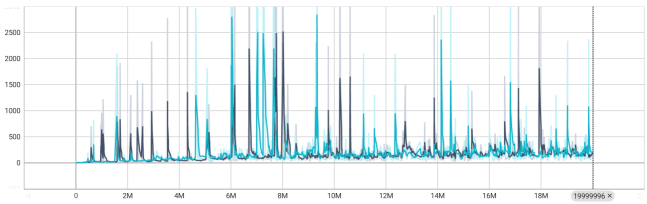


Figura 2: Curva `Loss/TD_Loss` de DQN.

2.3. Conclusiones de la Parte 1

DQN ha funcionado correctamente como punto de partida. Es capaz de aprender a caminar desde píxeles y lo hace de forma relativamente consistente entre ejecuciones, lo cual no es trivial en este tipo de problema. Sin embargo, el rendimiento alcanzado es moderado y el aprendizaje se vuelve inestable en fases avanzadas, como se ve en las oscilaciones en la recompensa y el comportamiento ruidoso de la pérdida TD.

En conclusión, podemos ver que DQN demuestra ser capaz de aprender una política útil en este entorno, pero con un rendimiento limitado y bastante variabilidad, mostrando que es un buen baseline, pero que el rendimiento y la estabilidad son bastante mejorables.

3. Parte 2: Locomoción con Rainbow DQN

3.1. Metodología

Para la comparación con Rainbow see mantienen exactamente las mismas decisiones de entorno que en DQN: mismo `Walker2d-v4`, misma discretización, misma observación visual y misma recompensa. Esto era imprescindible para poder atribuir las diferencias observadas al algoritmo y no a cambios en el entorno experimental.

Rainbow se entrenó con la configuración completa que aparece en el repositorio y que sigue la idea del trabajo de Hessel et al. [2]: Double DQN, arquitectura *dueling*, PER, *Noisy Nets*, *n-step returns* con $n = 3$ y aprendizaje distribucional C51 con 51 átomos y soporte $[-10, 1200]$. La red convolucional base es prácticamente la misma que la de DQN, pero el output cambia de forma importante. En lugar de devolver directamente valores Q, Rainbow aprende una distribución de retornos por acción y obtiene su valor esperado. Además, la exploración no se basa en ε-greedy, sino principalmente en el ruido de las capas *Noisy Linear*.

Un aspecto importante de esta parte es que Rainbow requiere un ajuste fino de hiperparámetros bastante más cuidadoso que DQN. Al combinar muchos componentes (PER, distribución, n-step, ruido paramétrico, etc.), hay varias configuraciones que influyen de forma muy significativa en el entrenamiento, y pequeñas variaciones pueden afectar mucho al resultado final. Por ello, fue necesario dedicar tiempo a ajustar estos parámetros hasta obtener un comportamiento estable.

La Tabla 3 resume los parámetros más importantes de esta configuración. Entre ellos destacan especialmente los relacionados con PER y con la parte distribucional, ya que fueron dos de los bloques más sensibles durante el entrenamiento y, como se verá más adelante, también dos de los que más influyen en el resultado final.

Cuadro 3: Parámetros específicos de Rainbow.

Parámetro	Valor
Double DQN	Sí
Dueling	Sí
PER	Sí
α_{PER}	0.6
β_0	0.4
β-frames	2 000 000
PER ϵ	10^{-6}
Noisy Nets	Sí
σ_0 inicial	0.5
n-step	3
γ_n	0.99
Distribucional	Sí
Número de átomos	51
$V_{\text{mín}}$	-10.0
$V_{\text{máx}}$	1200.0

También merece la pena señalar que Rainbow no añade solo un componente nuevo, sino varios mecanismos que interactúan entre sí. Precisamente por eso, la configuración de la tabla no debe verse como una lista de detalles aislados, sino como un conjunto de decisiones que en la práctica condicionan bastante la estabilidad del aprendizaje.

3.2. Resultados y discusión

La ejecución de Rainbow alcanzó los 20 millones de pasos y obtuvo un valor de `Reward/MeanWindow` de 3247.21, muy por encima del mejor resultado de DQN. Como se observa en la Figura 3, esta mejora no aparece solo al final, sino desde las primeras fases del entrenamiento. Rainbow aprende más rápido y alcanza niveles de recompensa altos mucho antes que DQN, manteniéndose además con una tendencia creciente durante más tiempo.

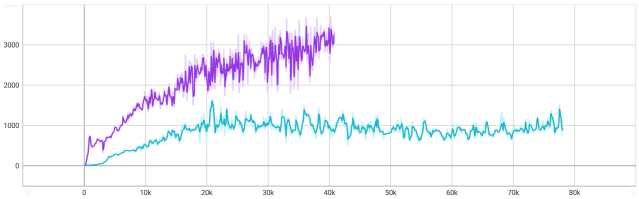
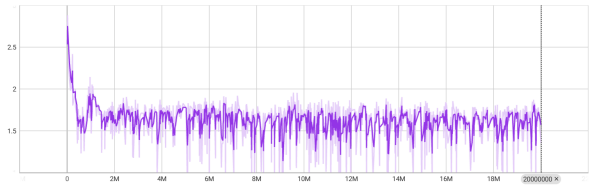


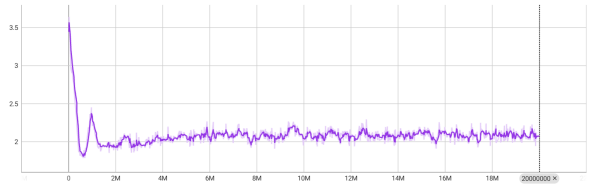
Figura 3: Comparación entre DQN y Rainbow en `Reward/MeanWindow`.

En cambio, como ya hemos visto antes, DQN muestra una dinámica más limitada: tras una mejora inicial, la curva se estabiliza en una zona bastante más baja y que posteriormente crece menos también. Esto nos sugiere que Rainbow no solo aprende antes, sino que también es capaz de seguir mejorando la política más allá del punto en el que DQN se estanca.

Otro aspecto que llama la atención es que la curva de Rainbow se corta antes en el eje horizontal. Esto no se debe a menos pasos de entrenamiento (el número de pasos era fijo y el mismo para ambos algoritmos), sino a que sus episodios son más largos, ya que el agente consigue mantenerse vivo durante más tiempo. Es decir, Rainbow no solo obtiene más recompensa, sino que también genera trayectorias más estables dentro de cada episodio.



(a) `Loss/DistributionalLoss`



(b) `Loss/MeanSampleLoss`

Figura 4: Evolución de las pérdidas en Rainbow durante el entrenamiento.

En cuanto a las pérdidas (Figura 4), tanto la `DistributionalLoss` como la `MeanSampleLoss` muestran un comportamiento bastante estable. Tras una caída inicial rápida, ambas se estabilizan en valores bajos (1.57 y 2.07 respectivamente), con oscilaciones moderadas pero sin mostrar una tendencia divergente. A diferencia de la TD loss de DQN, aquí no aparecen picos extremos, lo que quiere decir que el proceso de optimización es más controlado.

3.3. Conclusiones de la Parte 2

Rainbow consigue un rendimiento claramente superior al de DQN en este entorno. No solo alcanza recompensas más altas, sino que lo hace más rápido y con episodios más largos. Sin embargo, para que el método funcione correctamente, es necesario hacer un ajuste fino de hiperparámetros, ya que existen varias configuraciones sensibles que pueden afectar de forma importante al entrenamiento.

En este sentido, Rainbow no debe interpretarse simplemente como una mejora directa sobre DQN, sino como un modelo con mayor capacidad, con varios componentes

añadidos que interactúan entre sí, cuyo buen rendimiento depende en gran medida de una configuración adecuada.

4. Parte 3: Estudio de ablación en Rainbow DQN

4.1. Metodología

La tercera parte de la práctica consistía en analizar el efecto de dos mejoras concretas de Rainbow. En nuestro caso elegimos *Distributional RL* y PER, porque ambas parecían especialmente relevantes en un entorno visual donde la recompensa puede ser ruidosa y el poder aprovechar las transiciones del *buffer* puede ser una diferencia importante.

A partir de la versión completa de Rainbow se entrenaron tres variantes:

1. Rainbow sin componente distribucional (`noDIST`).
2. Rainbow sin PER (`noPER`).
3. Rainbow sin PER y sin componente distribucional (`noPER_noDIST`).

Aunque la práctica pedía estudiar el efecto de dos mejoras, nos pareció útil incluir también la ablación conjunta, porque permite ver si el deterioro del rendimiento es progresivo o si hay un efecto más fuerte cuando se eliminan ambos componentes a la vez. En todos los casos se mantuvieron fijos el entorno, la discretización, la recompensa, el preprocesado visual y el resto de hiperparámetros.

Hay un detalle que aclarar sobre las losses. En el script de entrenamiento, la etiqueta de TensorBoard `Loss/DistributionalLoss` se utiliza también cuando la variante sin componente distribucional pasa a una pérdida TD estándar. Por tanto, en las variantes `noDIST` y `noPER_noDIST`, esa curva no representa estrictamente una pérdida distribucional, pero sigue siendo útil como indicador de estabilidad del entrenamiento.

4.2. Resultados y discusión

Los resultados de la ablación muestran una separación muy clara entre las configuraciones. La Figura 5 permite ver directamente el impacto en el rendimiento del agente. Las variantes que mantienen el componente distribucional (Rainbow completo y `noPER`) alcanzan recompensas muy altas, superiores a 2000 y llegando en el mejor caso a más de 3000. Además, ambas mejoran rápidamente al inicio y ,mejoran de forma sostenida durante gran parte del entrenamiento.

En cambio, las configuraciones sin componente distribucional (`noDIST` y `noPER_noDIST`) tienen un comportamiento completamente distinto. Tras una pequeña mejora inicial, ambas se estancan rápidamente en valores muy bajos, en torno a 300–500, sin mejorar casi en lo que quedaba de entrenamiento. Es decir, el agente es incapaz de aprender una política de movimiento efectiva si le quitamos ese componente. Quedandose en un rendimiento más pobre que el de la DQN base.

Comparando directamente las variantes, se observa que eliminar PER produce que el rendimiento se degrade un poco ($\sim$ 3200 pasa a $\sim$ 2600), pero mantiene una política claramente mejor que la de DQN. En cambio, eliminar el componente distribucional provoca una caída muy fuerte ($\sim$ 3200 pasa a $\sim$ 400), prácticamente anulando la ventaja de Rainbow. Además, la diferencia entre `noDIST` y `noPER_noDIST` es pequeña, lo que sugiere que, una vez eliminado el componente distribucional, el efecto de PER pasa a ser marginal.

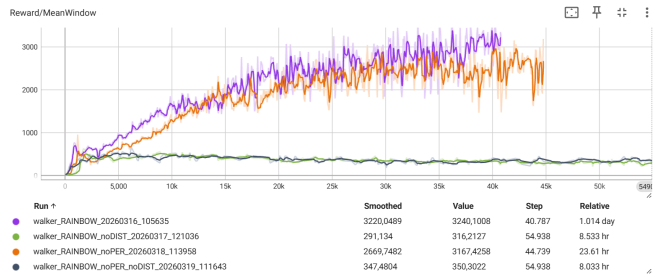
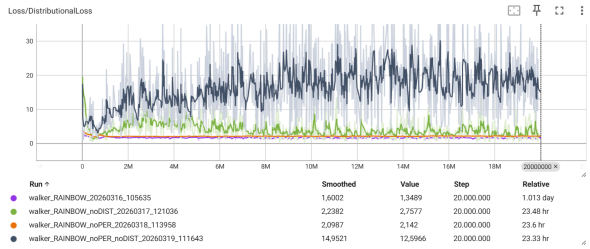


Figura 5: Comparación de `Reward/MeanWindow` entre Rainbow completo y las variantes ablacionadas.

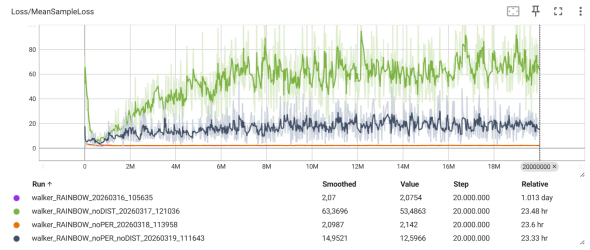
La lectura de las losses refuerza esta interpretación. En la Figura 6 se observa que Rainbow completo y la variante `noPER` mantienen valores bajos y muy estables tanto en `MeanSampleLoss` como en la pérdida asociada al entrenamiento distribucional, en torno a 2. Esto indica un proceso de optimización que está bastante controlado.

Por el contrario, la variante `noDIST` entra en un régimen mucho más inestable. La `MeanSampleLoss` crece hasta valores muy altos (del orden de 50–70) y presenta oscilaciones fuertes durante todo el entrenamiento. Este comportamiento coincide con lo visto en la evolución de la recompensa y sugiere que la señal de aprendizaje se vuelve mucho más ruidosa al eliminar el componente distribucional.

La configuración `noPER_noDIST` también empeora claramente respecto a Rainbow completo, aunque queda en una situación intermedia, con pérdidas más altas que las configuraciones buenas pero sin llegar al nivel extremo de `noDIST`. En conjunto, las pérdidas muestran dos regímenes bien diferenciados: uno estable (con componente distribucional) y otro claramente más ruidoso e ineficiente (sin él).



(a) `Loss/DistributionalLoss`



(b) `Loss/MeanSampleLoss`

Figura 6: Comparación de pérdidas en el estudio de ablación.

Este resultado es especialmente interesante porque no todas las mejoras de Rainbow tienen el mismo peso en este *setup*. Aunque PER suele destacarse por mejorar la eficiencia muestral, aquí su efecto es relativamente limitado en comparación con el del componente distribucional. Una posible explicación es que, en un entorno visual con dinámica compleja, modelar una distribución de retornos en lugar de un único valor esperado proporciona una señal de entrenamiento más rica y ayuda a estabilizar el aprendizaje.

PER sí aporta una mejora, pero más bien como un refinamiento. Su eliminación no rompe el aprendizaje, mientras que la eliminación del componente distribucional sí lo hace. Por tanto, la mejora de Rainbow en este experimento no se reparte de forma homogénea entre sus componentes, sino que está dominada claramente por uno de ellos.

Cuadro 4: Resumen de resultados finales.

Modelo	MeanWindow final
DQN run 1	773.57
DQN run 2	869.45
Rainbow completo	3247.21
Rainbow noDIST	417.16
Rainbow noPER	2790.91
Rainbow noPER_noDIST	357.02

La Tabla 4 resume bastante bien lo que ya se había visto en las curvas: hay una diferencia muy clara entre las configuraciones con y sin componente distribucional. Mientras que Rainbow completo y `noPER` se mantienen en niveles altos de recompensa, las variantes sin distribución se quedan en valores muy bajos, incluso por debajo de DQN.

Cuadro 5: Resumen de pérdidas finales.

Modelo	Loss 1	Loss 2
DQN run 1	TD: 190.59	–
DQN run 2	TD: 200.11	–
Rainbow completo	1.57	2.07
Rainbow noDIST	2.26	62.13
Rainbow noPER	2.10	2.10
Rainbow noPER_noDIST	14.25	14.25

Por otro lado, la Tabla 5 refuerza esta misma idea desde el punto de vista de la optimización. Las configuraciones que funcionan bien mantienen pérdidas bajas y estables, mientras que la variante sin componente distribucional presenta valores mucho más altos, lo que encaja con el comportamiento más inestable que hemos observado durante el entrenamiento.

4.3. Conclusiones de la Parte 3

El estudio de ablación muestra de forma bastante clara que no todas las mejoras de Rainbow tienen el mismo impacto en este problema. El componente distribucional es el más importante de los dos analizados: quitarlo provoca una caída muy fuerte del rendimiento y un aumento claro de la inestabilidad en el entrenamiento.

PER, por su parte, también aporta una mejora, pero su efecto es más reducido. Quitarlo baja el rendimiento, pero no impide que el agente aprenda una política mucho mejor que la de DQN. En cambio, al eliminar el componente distribucional, el comportamiento del agente cambia completamente y deja de aprovechar el entorno de forma efectiva.

En conjunto, los resultados indican que la principal ventaja de Rainbow en este experimento no se debe a la suma uniforme de todas sus mejoras, sino principalmente al uso del aprendizaje distribucional, que parece ser clave para mantener una señal de entrenamiento estable y conseguir un rendimiento alto en este entorno.

5. Conclusiones generales

Esta práctica nos ha servido para construir y analizar un pipeline completo de Deep Reinforcement Learning aplicado a locomoción visual en MuJoCo usando value based methods. Una de las ideas más claras que nos deja el trabajo es que, en este tipo de problemas, el resultado final no depende solo del algoritmo. Aspectos como la discretización del espacio de acciones, el preprocesado de las imágenes o la forma concreta en la que implementas la recompensa influyen mucho en la estabilidad del entrenamiento y en lo que el agente es capaz de aprender.

La DQN estándar ha cumplido bien su papel como baseline. Ha conseguido aprender una forma de moverse razonable a partir de píxeles y lo ha hecho con una dinámica bastante consistente entre ejecuciones. Aun así, su rendimiento final se queda en un nivel medio y da la impresión de que, tras una fase inicial de mejora, el aprendizaje se estanca bastante. Esto nos lleva a pensar que el pipeline estaba bien planteado, pero que la capacidad de DQN se queda algo corta para aprovechar del todo el entorno.

Rainbow, por su parte, ha supuesto una mejora clara tanto en rendimiento como en velocidad de aprendizaje. Cuando está bien ajustado, no solo alcanza recompensas mucho más altas, sino que además lo hace antes y mantiene episodios más largos. Sin embargo, también hemos visto que esa mejora no sale gratis: Rainbow es bastante más sensible a los hiperparámetros y requiere más fine tuning para funcionar de forma estable. En ese sentido, la práctica también deja claro que un algoritmo más potente suele venir acompañado de una mayor dificultad práctica a la hora de entrenarlo.

Probablemente la parte más interesante del trabajo ha sido el estudio de ablación. Ahí se ve bastante bien que no todas las mejoras de Rainbow tienen el mismo impacto en el rendimiento final. El componente distribucional ha resultado ser claramente el más importante en nuestro caso: al quitarlo, el rendimiento cae mucho y el entrenamiento se vuelve bastante más inestable. En cambio, al eliminar PER también se degrada un poco, pero bastante menos, y el agente sigue siendo capaz de aprender una política buena.

En conjunto, los resultados apuntan a que la ventaja de Rainbow en este experimento no viene de sumar muchas mejoras pequeñas sin más, sino sobre todo de contar con una parte distribucional que hace que la señal de aprendizaje sea más útil y más estable en un entorno complejo y observado desde píxeles. PER ayuda, pero parece que su papel es más de refinamiento que de componente esencial.

Como conclusión general, Rainbow es capaz de superar a DQN en este tipo de tareas con bastante margen, pero su buen funcionamiento depende mucho de que el entrenamiento esté bien ajustado y de que todo el pipeline sea coherente. La principal lección que nos llevamos de la práctica no es solo que un algoritmo sea mejor que otro, sino que en Deep RL los detalles de implementación y las decisiones de diseño tienen un peso enorme en los resultados finales.

Trabajo futuro. Como continuación natural, sería interesante repetir los experimentos con varias semillas para poder medir mejor la variabilidad entre ejecuciones, especialmente en el caso de Rainbow. También sería útil analizar el efecto de otros componentes del algoritmo o probar este mismo pipeline en distintos entornos, para ver hasta qué punto las conclusiones obtenidas aquí se mantienen o dependen del problema concreto.

Referencias

[1] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra y Martin Riedmiller. *Playing Atari with Deep Reinforcement Learning.* arXiv preprint arXiv:1312.5602, 2013. DOI: https://doi.org/10.48550/arXiv.1312.5602.

[2] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar y David Silver. *Rainbow: Combining Improvements in Deep Reinforcement Learning.* arXiv preprint arXiv:1710.02298, 2017. DOI: https://doi.org/10.48550/arXiv.1710.02298.